

Operators in java

* Type promotion Rules

- byte short & char values are promoted to int
- If one operand is long, the whole expression will become long.
- If one operand is float, entire expression will become float
- If one operand is double, entire expression is double

* Operators

* Arithmetic operators

:- (+, -, *, /, %)

(+=, -=, *=, /=, %=
++, --)

* Relational Operators

==, !=, <, >, <=, >=

→ used in else if conditions

```
int a = 5;
```

```
int b = 10;
```

```
a == b; // false
```

a = b → a is assigned the value of b

* Bitwise Operator

→ Bit Manipulation

~~→ &, |, ^, ~, >>, <<~~

& and

~~and~~ | bitwise and

^ bitwise exclusive or

~ ~~bitwise~~ not (unary operator)

>> shift right

<< shift left

>>> shift right with zero fill, & so on....

(>>=, <<=, >>>=, |=, &=, ^=)

byte ~~int~~ a = 2;

byte ~~int~~ b = 3;

int c = a & b; $\rightarrow$ Integer

2 $\rightarrow$ 10

3 $\rightarrow$ 11

a = 0 0 0 0 0 0 1 0
b = 0 0 0 0 0 0 1 1 &

a	b	a & b	a b	a ^ b	~a	~b
0	0	0	0	0	1	1
0	1	0	1	1	1	0
1	0	0	1	1	0	1
1	1	1	1	0	0	0

Truth Table

C = C & 2; $\Rightarrow$ C &= 2;

Shift operators

left shift (<<)

byte b = 8;

b = b << 1;

b = b << 1

0 0 0 0 1 0 0 0 = 8
0 0 0 1 0 0 0 0 = 16
0 0 1 0 0 0 0 0 = 32

b << 1; $\Rightarrow$ b = b * 2;

b << 2; $\Rightarrow$ b = b * 2²

by going

on b becomes zero

at one time & becomes constant

$\ll$ or $\gg$ $\rightarrow$ (perform only on)

$\rightarrow$ int, long $\checkmark$

$\rightarrow$ byte, short $\times$

(if doing byte, short they get promoted to int)

~~int~~ int i = 1

then $i \ll 31$; Maxⁿ MSP

after that it does $i \ll (32 \% 32)$

i.e mod of 32

Right shift ($\gg$) : opp of left shift

$\ggg$ shift right with zeros
 $\rightarrow$ dumb shift

byte b = -128; $\underline{1} \ \underline{0} \ \underline{0} \ \underline{0} \ \underline{0} \ \underline{0} \ \underline{0} \ \underline{0} = -126$

$b \gg 1$; $\underline{1} \ \underline{1} \ \underline{0} \ \underline{0} \ \underline{0} \ \underline{0} \ \underline{0} \ \underline{0} = -64$

$b \ggg 1$; $\underline{0} \ \underline{1} \ \underline{0} \ \underline{0} \ \underline{0} \ \underline{0} \ \underline{0} \ \underline{0} = +64$

* Logical Operators

$\rightarrow$ $\&\&$ (AND)

$\|$ (OR)

! (NOT)

this doesn't work on bits but on expressions

int a = 5;

int b = 10;

int c = 15;

		AND	OR
True	false	False	True
false	True	False	True
True	True	True	False True
false	false	False	false

boolean and = $(a < b) \ \&\& \ (a < c)$

$\downarrow$
True

$\downarrow$
T

$\downarrow$
T

Short Circuit

A	B	A && B
F	F	F
F	T	F
T	F	F
T	T	T

A && B

↓

F

If any one is false then don't check B

because it will give false

& this concept is called Short circuit

& opposite goes for OR operation

Bitwise operators

int a = 5

b = 10

c = 10;

we can use bitwise operators here as well

d = (a < b) & (a < c)

but it does not do short circuiting

* Assignment operator

int a = 5;

int b = a; → value of a is assigned to b

* Ternary operator

→ used in conditionals

* operator precedence

`int c = a * b + d - f / g << 2;`

like in Maths we've BODMAS

in java we have precedence of operators which also follows BODMAS

lowest precedence is of assignment (`=`) operator;

highest precedence is of bracket (`()`);